

Memory and State Control for LLM-Driven NPC Dialogue: Scoped Dual-Tier Memory and Pre-Commit Invariant Validation under a Paired, Oracle-Audited Evaluation

Parth Mital*

Bhumika Singh

Pranesh Gopal

parth.mital2023@vitstudent.ac.in bhumika.2023a@vitstudent.ac.in pranesh.gopal2023@vitstudent.ac.in

Dr. Kalyanaraman P (Faculty)

pkalyanaraman@vit.ac.in

School of Computer Science and Engineering, Vellore Institute of Technology

October 2026

Abstract

Large language models (LLMs) let non-player characters (NPCs) hold open-ended conversations, but three problems limit their use in games: facts established early in a session are forgotten, model-proposed world changes break symbolic game rules, and per-turn cost grows with context. We study **OBSIDIAN FLASK**, an NPC dialogue engine that separates language generation from state control. It combines a dual-tier memory (an 8-turn buffer plus a 384-dimensional FAISS store whose retrieval is scoped to the current location and NPC, with a relaxation fallback) with an event-sourced world state in which every LLM-proposed mutation must pass a pre-commit validation barrier before a pure reducer applies it. The question is not whether the engine works in its own world, but whether its two mechanisms beat independent baselines and whether each contributes on its own. We evaluate it against rolling-context, flat vector-memory and generic retrieval-augmented agents and a full-history reference, in a 2×2 memory-by-control factorial with seven single-component ablations, on seven worlds from four families (8 to 128 locations, including worlds converted from the crowd-authored **LIGHT** corpus). The evaluation’s distinctive device is a paired three-way commit: every model output is committed through the barrier, through the reducer alone, and by direct writes, and the resulting states are audited by an invariant oracle that shares no code with the validator. In a 6.69-hour run with a self-hosted 7B backbone, the system recalled 87.7 [84.2, 91.0]% of facts 20 or more turns old on held-out worlds, against 0.0% for a rolling context of equal token budget, 63.9% for generic retrieval and 80.2% for full history. Its advantage over flat vector memory (84.9%) was not significant ($p = 0.298$), and it used more input tokens, so the pre-registered memory hypothesis is not supported. Scoping raised MRR@6 from 0.750 to 0.898. On identical outputs the barrier cut adversarial attack success from 16.2% to 2.9% with no false rejection of lawful requests and 0.039 ms of validation at the 95th percentile, but it missed unbounded currency gains (19.7% attack success) and raised dialogue-state desynchronisation from 3.7% to 18.7% of live turns. Event-log replay reproduced the live state in every session, whereas the application’s own load path lost the events after its last snapshot. Memory and control interacted on recall, so the mechanisms are not fully separable. The run used the third of eight pre-declared sample-size levels and automatic scoring, so the results are preliminary.

Keywords: large language models; non-player characters; long-term memory; retrieval-augmented generation; event sourcing; runtime verification; interactive fiction; evaluation methodology.

*Source code, benchmark and run outputs:

<https://github.com/parthmital/RAG-Driven-NPC-Narrative-Engine>

1 Introduction

Text-centric games mediate navigation, object interaction and character dialogue entirely through language. Unlike graphical engines, where physics enforces spatial constraints, a text world can only stay consistent through an explicit symbolic state. Games have traditionally driven NPCs with dialogue trees and finite-state machines, which obey the rules at negligible cost but restrict players to predefined choices. LLMs remove that restriction, and surveys of LLMs in games identify the coupling of generated text to game mechanics and state as an open problem [1].

Three failure modes stand in the way. First, *long-horizon forgetting*: facts that fall out of a bounded context window are lost, and facts inside a long context are used unevenly [2]; very long-term conversational memory remains weak even with retrieval [3, 4]. Second, *invariant violations*: LLMs are unreliable simulators of state transitions [5, 6] and lose narrative commitments over long interactions [7]; left unchecked they move players between unconnected rooms, invent items and grant unearned money. Third, *cost growth*: appending full history, or running several reflective LLM calls per turn [8, 9], raises tokens and latency per turn.

This paper evaluates an architecture that addresses all three by separating generation from state control (Section 4). Its components are individually standard: a turn buffer, dense retrieval, schema-constrained output, a rule validator, an append-only event log and a deterministic reducer. Our claim is therefore not that any component is new. The contribution is an evaluation that isolates what each mechanism buys, against independent baselines, with the costs it imposes measured rather than assumed.

Research questions. **RQ1** (memory): does scoped dual-tier memory improve long-range recall per prompt token over rolling-context, vector-memory and generic retrieval agents? **RQ2** (state control): does a pre-commit validation barrier over event-sourced state reduce committed invariant violations, and at what cost in false rejections and dialogue-state desynchronisation? **RQ3** (separability): do memory and control contribute independently? **RQ4** (efficiency): what latency and tokens does each component add? **RQ5** (generalisation): do the effects hold across worlds of different authorship, genre and scale, and across backbones?

Contributions.

1. A formal account of the engine as implemented: the world state, the event reducer and its replay and snapshot properties, the scoped retrieval operator with its post-filter limitation, and the validation predicate, including the coverage gaps found in the code (Sections 3 and 4).
2. A *paired three-way commit protocol* with an *independent oracle*: each LLM output is committed through the barrier, through the reducer alone and by direct writes, and the three post-states are audited by an eight-invariant oracle that imports nothing from the validator. Common random numbers make every control comparison exactly paired, and lawful *twins* of every attack measure false rejection (Section 5.6).
3. A multi-world benchmark (four families, seven worlds, long-horizon sessions of 50 to 200 turns, seven attack categories) and a single-notebook harness that reports recall, retrieval quality, violations, false rejection, desynchronisation, latency, tokens and replay consistency together, with cluster-bootstrap intervals and pre-registered hypotheses (Sections 5 and 6).
4. Empirical findings, including two unsupported hypotheses and four implementation defects that a validator-only evaluation would have missed: unbounded currency gains, a load path that discards the logged suffix, validator-accepted proposals that the naive writer turns into persistent corruption, and dialogue that contradicts rejected changes (Sections 7 and 9).

2 Related Work

Grounded dialogue and text games. LIGHT conditions dialogue on rooms, objects and personas in a crowd-written fantasy world [10], and goal-driven agents in LIGHT were trained with reinforcement learning over factorised speech and action spaces [11]. Neither keeps a persistent, validated world state over long sessions. TextWorld [12] and Jericho [13] provide formally specified or human-authored games whose engines enforce rules, but they parse a constrained command language and have no open NPC dialogue; TALES [14] gathers such games into a suite on which even the best LLM agents remain weak. Callison-Burch et al. [15] framed Dungeons and Dragons as a dialogue challenge including state prediction, and STORIUM anchors collaborative stories with structured cards [16].

Memory for LLM agents. Generative Agents combine a memory stream, reflection and scored retrieval [8]; Reflexion stores verbal self-reflections across trials [9]; MemGPT pages information between a bounded context and archival storage through LLM-invoked functions [17]. Later systems add forgetting curves [18], knowledge-graph indexing [19], self-organising linked notes [20] and LLM-driven consolidation [21]. Benchmarks show that long-term memory is far from solved [3, 4], and surveys organise the design space [22, 23]. These systems store natural-language memories and most spend extra LLM calls per write; none binds memory to a validated symbolic state, and none is compared at a matched token budget in a game setting.

Retrieval and long context. Retrieval-augmented generation conditions generation on passages from a dense index [24]; generic RAG ranks by similarity alone and has no notion of a turn’s spatial or social scope. Extending the context is not a substitute: models use the middle of long contexts poorly [2]. ReAct keeps a growing scratchpad of observations and actions [25]. Scoping by metadata is a filtered nearest-neighbour query; post-filtering a global top- k is known to lose in-scope neighbours, which motivates filter-aware indices [26]. We analyse exactly this effect in the engine’s retrieval (Proposition 2).

State tracking and verifier-gated generation. LLMs track entities imperfectly [6] and are unreliable text-world simulators [5]. The LLM-Modulo position argues that LLM proposals should be checked by external model-based verifiers [27]; grammar-constrained decoding guarantees syntactic well-formedness but not semantic legality [28]. Prompt injection shows that model output cannot be trusted to obey instructions in its context [29]. Event sourcing, where state is a fold over an append-only log, is a standard pattern for auditable systems [30].

LLM NPCs in games, 2024 to 2026. PANGeA generates RPG content with a memory system and a validator in which the LLM itself judges player input against design rules [31]. Autoregressive state-tracking prompting constrains in-game trading by having the model report a state label, with arithmetic done in post-processing [32]. Recent preprints study modular memory for small persona models on consumer hardware [33], role-sensitive prompt scaffolding [34], incremental cache maintenance for long-lived NPCs [35], and explicit world reconstruction from narratives [36]. NCP-Bench measures how interactive narratives drift under user interventions and finds fact conflicts dominate failures [7].

Positioning. Table 1 contrasts representative systems. The present work is, to our knowledge, the first to evaluate symbolic pre-commit validation of LLM-proposed game mutations with an oracle independent of the validator, paired on identical model outputs, together with memory scoping, replay and cost in one factorial design. We make no claim of novelty for the individual components.

Table 1: Positioning against representative prior work. “Indep. audit” means violations are scored by a checker that does not share code with the component being evaluated. Entries describe the cited papers’ own evaluations.

Work	Memory	World state	Pre-commit check	Indep. audit	Replay
LIGHT [10]	episode context	text descriptions	action checks	–	–
Generative Agents [8]	scored memory stream	sandbox tree	–	–	–
MemGPT [17]	LLM-paged archive	free text	–	–	–
A-Mem [20]	linked notes	none	–	–	–
TextWorld/Jericho	game engine	formal state	parser and rules	–	engine
PANGeA [31]	memory system	game level data	LLM judge	–	–
ASTP [32]	dialogue context	state label	post-processing	–	–
This work	scoped buffer + FAISS	event-sourced, typed	symbolic validator	yes (8 invariants)	yes

3 Formal Model

This section states the model the code implements; module paths refer to [Backend/](#) at the evaluated commit.

3.1 World state

Definition 1 (World state). A world state is a tuple $S = (\mathcal{L}, A, \mathcal{N}, \mathcal{O}, \pi, \iota, c, R, J, t)$ where $\mathcal{L}$ is the set of locations with directed adjacency $A \subseteq \mathcal{L} \times \mathcal{L}$ (the `connected_to` lists); $\mathcal{N}$ is the set of NPCs, each with a location, persona, knowledge, conditional secrets and an `alive` flag; $\mathcal{O}$ is the set of objects, each with a location in $\mathcal{L} \cup \{\perp\}$ and an owner; $\pi \in \mathcal{L}$ is the player’s location, ι the inventory and $c \in \mathbb{Z}_{\geq 0}$ the currency; $R : \mathcal{N} \times (\mathcal{N} \cup \{\text{player}\}) \rightarrow [-100, 100]$ is the trust map; J the journal; and t the turn ([schemas/world_state.py](#)).

The seed S_0 fixes the canonical entity sets $\mathcal{L}_0, \mathcal{N}_0, \mathcal{O}_0$. The field `active_npc_id` is part of the stored state but is set outside the event log (Section 4.1).

3.2 Events, reduction and replay

An event is $e = (id, t, \tau, p, ts)$ with type τ from 14 types ([schemas/events.py](#)) and payload p . The reducer δ ([core/reducer.py](#)) deep-copies the state and applies one event; it ignores moves to unknown locations, clamps trust to $[-100, 100]$, floors currency at zero, and ignores an NPC `location_id` change to a non-canonical location. State after T events is

$$S_T = \text{foldl}(\delta, S_0, [e_1, \dots, e_T]), \quad S_i = \delta(S_{i-1}, e_i). \quad (1)$$

Events are appended to SQLite with autoincrement ids, and every $\Delta = 16$ turns a snapshot (S_k, id_k) is written, where id_k is the last event id ([graph/definition.py](#)).

Proposition 1 (Replay and snapshot equivalence). *If (i) δ is a function of (S, e) only and (ii) every change to the hashed part of the live state is made by applying a logged event with δ , then for any snapshot taken after event k ,*

$$\text{foldl}(\delta, S_0, e_{1..T}) = \text{foldl}(\delta, \text{foldl}(\delta, S_0, e_{1..k}), e_{k+1..T}) = \text{foldl}(\delta, S_k, e_{k+1..T}).$$

Proof. The first equality is the definition of a left fold over a concatenated list; the second substitutes the snapshot. Condition (i) holds in the code because the only time-dependent value the reducer writes, a journal entry’s timestamp, is read from the event, which is persisted with it. Condition (ii) fails for `active_npc_id`, which is why replay hashes exclude it. $\square$

Recovery from the latest snapshot therefore costs $O(|S| + r)$ with r the number of events after it, against $O(|S| + T)$ for a full fold. The application’s load path (`session/manager.py`, `load_session`) loads (S_k, id_k) and *discards* id_k , returning S_k instead of `foldl( $\delta, S_k, e_{k+1..T}$ )`: every event after the last snapshot is lost unless a per-turn auto-save happened to succeed. Section 7.6 measures the consequence.

3.3 Proposals and the validation barrier

Each turn the LLM returns one JSON object validated by a Pydantic schema (`schemas/llm_output.py`): dialogue, narration, a one-sentence memory summary, a list of proposals $P_t = [(\tau_j, p_j)]$ and a list of proposed new entities. The barrier (`game/validator.py`) evaluates a predicate $V(\tau, p, S_t)$ for each proposal *independently against the pre-turn state* S_t , and commits

$$E_t = [e(\tau_j, p_j) : (\tau_j, p_j) \in P_t, V(\tau_j, p_j, S_t)], \quad S_{t+1} = \text{foldl}(\delta, S_t, [e_{\text{speech}}] ++ E_t). \quad (2)$$

V requires: a move’s destination is canonical and in the current location’s adjacency list; a trust change names a canonical NPC with $|\Delta| \leq 20$; a taken object is canonical and lies at the player’s location; a dropped object and its target location are canonical and, if dropped by the player, carried; state and flag updates name a canonical target and a key; a journal entry has content; a currency delta is numeric and, if negative, $|\Delta| \leq c$. Any non-empty `new_entities` list is rejected and logged. Rejected proposals are dropped and the turn continues; dialogue is not regenerated.

3.4 Invariants audited by the oracle

The evaluation’s oracle (notebook cell 36) re-derives facts from $(S_{\text{pre}}, S_{\text{post}}, \text{committed mutations})$ and checks eight invariants, written from the specification rather than the validator. With $\mathcal{L}_0, \mathcal{N}_0, \mathcal{O}_0$ from the seed and $\mathcal{A}$ the *undirected* closure of the seed adjacency:

O1 canonicity

post-state entity sets equal the seed’s; the player, inventory and NPC locations reference canonical ids.

O2 conservation

each object is in exactly one place: a location, the inventory, or an NPC’s possession.

O3 adjacency

$\pi_{\text{post}} = \pi_{\text{pre}}$ OR $(\pi_{\text{pre}}, \pi_{\text{post}}) \in \mathcal{A}$.

O4 theft/duplication

a newly held object was at the player’s location, on the floor or held by an NPC who is there, and is held once.

O5 relocation

objects move only to the player’s location; NPCs move only along $\mathcal{A}$.

O6 trust bound

$|R_{\text{post}}(n, \text{player}) - R_{\text{pre}}(n, \text{player})| \leq 20$ for every NPC.

O7 solvency

total spending in the turn $\leq c_{\text{pre}}$ and $c_{\text{post}} \geq 0$.

O8 unearned currency

$c_{\text{post}} - c_{\text{pre}} \leq 50$.

O8 encodes a design bound the validator does not implement; it is the oracle’s only rule with no counterpart in V , and it exposes the gap reported in Section 7.3.

3.5 Scoped dual-tier memory

Long-term memory is a set of entries $m_i = (\mathbf{v}_i, x_i, t_i, \ell_i, n_i)$: a unit vector, the text shown to the model, the turn, a location and an NPC. On commit, $\mathbf{v}_i$ is the embedding of the *player input* under [all-MiniLM-L12-v2](#) [37, 38] ($d = 384$, L2-normalised), x_i is the model’s memory summary (or the truncated input when absent), ℓ_i is the player’s location after the turn’s events and n_i the active NPC. The index is an exact inner-product FAISS index [39], so $\mathbf{q}^\top \mathbf{v}_i$ is cosine similarity. For query $\mathbf{q}$, current location ℓ and NPC n , let $\text{top}_m(\mathbf{q})$ be the m most similar entries in descending order and define the scope predicate

$$\phi_{\ell,n}(m_i) = (\ell_i = \emptyset \vee \ell_i = \ell) \wedge (n_i = \emptyset \vee n_i = n), \quad (3)$$

where empty metadata acts as a wildcard. Scoped retrieval with over-fetch factor $f = 5$ and threshold $\theta = 3$ is

$$\mathcal{R}(\mathbf{q}) = \begin{cases} \text{first}_k[m \in \text{top}_{\min(fk, |M|)}(\mathbf{q}) : \phi_{\ell,n}(m)] & \text{if this list has } \geq \theta \text{ entries,} \\ \text{top}_k(\mathbf{q}) & \text{otherwise (relaxation fallback),} \end{cases} \quad (4)$$

with $k = 6$. The fallback *replaces* the scoped list rather than extending it. The short-term tier is a ring buffer of the last $B_s = 8$ turns. When $|M| > 1000$ the store keeps its newest 500 entries.

Proposition 2 (Post-filter reachability). *An in-scope entry m can be returned by the scoped branch of (4) only if its similarity rank among all entries is at most $fk = 30$. Hence scoped recall can fall below the recall of an exact filtered search whenever more than $fk - 1$ out-of-scope entries are more similar to $\mathbf{q}$ than m .*

Proof. The scoped branch filters $\text{top}_{fk}(\mathbf{q})$; entries outside it are never inspected. $\square$

The prompt contains a fixed system block, the canonical state of the current location and active NPC, player identity, the retrieved memory texts, the buffer and the query, capped at 16,384 characters by dropping memories first and history second ([llm/prompt_builder.py](#)).

4 System Architecture

Figure 1 shows the four tiers. The presentation tier (React, TypeScript, Zustand) streams turns over a WebSocket; the orchestration tier (FastAPI with per-session async locks) runs a compiled eight-node LangGraph `StateGraph`; validation and storage are described above. In the shipped game the LLM is a hosted endpoint (Groq, `openai/gpt-oss-120b`, temperature 0.35, [Backend/config.py](#)); in the evaluation it is a self-hosted model (Section 6). Algorithm 1 gives one turn in pseudocode and Algorithm 2 its scoped retrieval step.

Algorithm 1 One turn of the engine (nodes 1 to 8 of Figure 1)

Require: player input u , active NPC n , state S_t , store M , buffer H , log $\mathcal{E}$

- 1: $u \leftarrow \text{TRIM}(u)$; **if** $u = \varepsilon$ **then** $u \leftarrow \text{“(silence)”}$
 - 2: $\mathbf{q} \leftarrow \text{EMBED}(u)$; $\mathcal{R} \leftarrow \text{SCOPEDRETRIEVE}(\mathbf{q}, \pi(S_t), n)$ ▷ Algorithm 2
 - 3: $x \leftarrow \text{BUILDPROMPT}(S_t, n, u, \mathcal{R}, H)$ ▷ memories trimmed first
 - 4: $y \leftarrow \text{LLM}(x)$; $o \leftarrow \text{PARSEJSON}(y)$ ▷ on failure: y becomes dialogue, no proposals
 - 5: $E_t \leftarrow [e(\tau, p) : (\tau, p) \in o.\text{proposals}, V(\tau, p, S_t)]$ ▷ Equation (2); entity creation rejected
 - 6: append e_{speech} and E_t to $\mathcal{E}$; $S_{t+1} \leftarrow \text{foldl}(\delta, S_t, E_t)$
 - 7: $H.\text{push}(u, o.\text{dialogue}, n)$; $M.\text{add}(\mathbf{q}, o.\text{summary}, t+1, \pi(S_{t+1}), n)$
 - 8: **if** $|M| > 1000$ **then** keep the newest 500 entries of M
 - 9: **if** $(t+1) \bmod 16 = 0$ **then** write snapshot $(S_{t+1}, \text{maxid}(\mathcal{E}))$ and save M
 - 10: **return** dialogue, narration, S_{t+1}
-

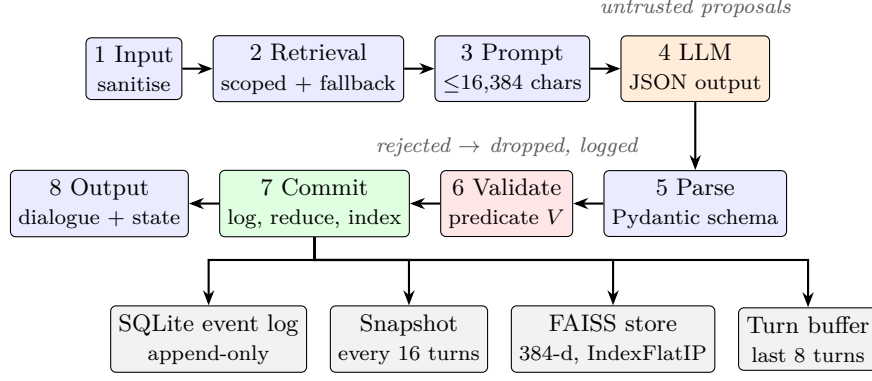


Figure 1: Turn pipeline of OBSIDIAN FLASK ([graph/definition.py](#)). The LLM output is untrusted: proposals reach the world state only through the validation barrier (node 6) and the pure reducer inside the commit node (node 7). The commit node writes the four stores; on later turns, retrieval (node 2) reads the FAISS store and prompt assembly (node 3) reads the turn buffer.

Algorithm 2 SCOPEDRETRIEVE (`make_node_retrieval`, `FAISSMemory.search`)

Require: query $\mathbf{q}$, location ℓ , NPC n , $k = 6$, $f = 5$, $\theta = 3$

- 1: $C \leftarrow$ exact inner-product top-min($fk, |M|$) of $\mathbf{q}$ over M
 - 2: $R \leftarrow []$
 - 3: **for** $m \in C$ in descending similarity **do**
 - 4: **if** $\phi_{\ell,n}(m)$ **then** append m to R ; **if** $|R| = k$ **then break**
 - 5: **if** $|R| < \theta$ **then** $R \leftarrow$ first k entries of C $\triangleright$ relaxation fallback, unscoped
 - 6: **return** R
-

Complexity. With $|M| \leq 1000$ entries, $d = 384$ and p proposals per turn: exact search is $O(|M|d)$; validation is $O(p)$ dictionary and adjacency lookups; reduction is $O(p|S|)$ because each event deep-copies the state; recovery from the latest snapshot is $O(|S| + r)$ with r the events since it. Measured costs are in Section 7.7.

4.1 Coverage gaps in the implementation

Reading the code against the specification, and the evaluation itself, exposed the following gaps at the evaluated commit. They are stated here so that the evaluation can measure them.

- G1. **Currency gains are unbounded:** only negative deltas are checked.
- G2. **Per-proposal, not per-turn, checking:** every proposal is validated against S_t , so several spends that each fit the balance can jointly overspend (the reducer then floors the balance at zero), and chained effects within one turn are not re-checked.
- G3. **NPC drops and NPC relocation:** drops by NPCs do not check possession, and `NPC_STATE_CHANGED` with key `location_id` is accepted for any value; the reducer, not the validator, ignores a non-canonical NPC location.
- G4. **Prompt and validator disagree on trust:** the prompt advertises deltas in $[-10, 10]$, the validator allows ± 20 .
- G5. **Secrets never reach the model:** NPC secrets are not placed in the prompt, so the trust-60 reveal rule cannot be honoured and leak rates do not measure secret keeping.
- G6. **Load path ignores the logged suffix,** and `active_npc_id` is set outside the log (Proposition 1).
- G7. **Bare JSON values:** at the evaluated commit the JSON extractor could return a string or list, which the parse node did not handle (3 of 3,276 live turns raised). This was fixed after

the evaluation (commit [a9767d5](#)); no other backend file changed.

- G8. **Structural, not narrative, legality:** an in-bounds but unjustified trust change passes, and rejection does not rewrite the NPC’s dialogue.

5 Benchmark and Experimental Setup

5.1 World suite

A single hand-made world cannot support generalisation claims, and a benchmark written by the system’s authors risks being tuned to it. The suite therefore has four families (Table 2 and Figure 2): W1, the in-house dark-fantasy development world ([Backend/game/world_seed.json](#)); W2a to W2c, converted by a deterministic seeded script from the crowd-written [LIGHT environment file](#) [10] (CC BY-NC 4.0), with LIGHT’s fragmented room graph joined by a seeded spanning tree and two templated secrets per NPC built from words that occur nowhere else in the world; W3, a 24-location science-fiction station authored in the notebook from a written specification before any system run; and W4-32 and W4-128, procedural graphs (random spanning tree plus about 30% extra edges) with templated names. No system setting was tuned on W2 to W4. W3 is held out from tuning but was *not* independently authored.

Table 2: World suite as built for the run ([metrics/world_stats.csv](#)). “Join” counts spanning-tree edges added to connect LIGHT’s room graph.

World	Family	Locations	NPCs	Objects	Rules	Edges	Diameter	Secrets	Join
W1	in-house (in-distribution)	8	4	4	7	7	4	8	0
W2a	LIGHT-derived	16	6	7	5	15	8	12	14
W2b	LIGHT-derived	24	6	9	5	23	9	12	22
W2c	LIGHT-derived	32	6	12	5	31	9	12	27
W3	held-out genre	24	6	8	6	27	4	12	0
W4-32	procedural	32	8	16	5	39	9	16	0
W4-128	procedural	128	16	64	5	163	12	32	0

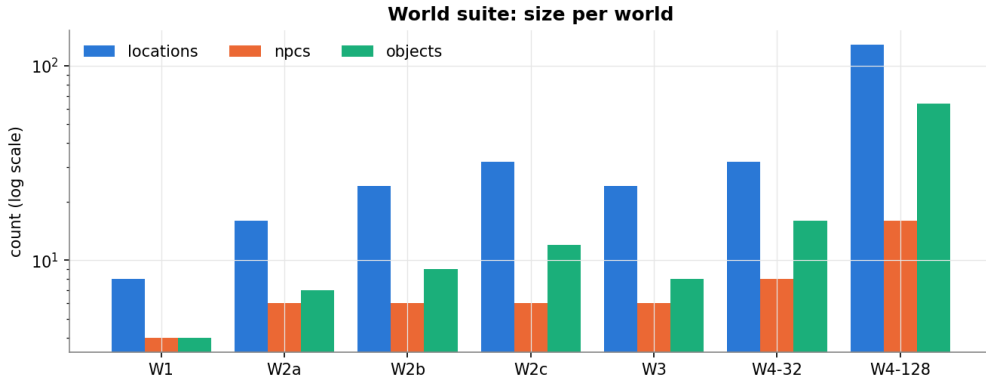


Figure 2: Size of each world (log scale): locations, NPCs and objects. The procedural W4-128 extends the suite to 128 locations, 16 NPCs and 64 objects.

5.2 Teacher-forced long-horizon recall sessions

A session is a fixed script of $H \in \{50, 100, 200\}$ player turns. For each fact age $a \in \{5, 10, 20, 40, 80, 160\}$ with $a < H - 1$, two facts are *planted* with a randomly chosen NPC and *probed* a turns later with the same NPC, using 15 attribute templates (meeting place, password, debt, and so on). Each fact has a paraphrased *distractor* with a different value told

to a different NPC, placed between plant and probe where possible. Values come from pools of invented or uncommon words; any value whose key word occurs anywhere in a world’s text is removed, so a correct answer cannot come from lore. A probe is correct when the gold key word appears as a whole word in the reply or narration, and *confused* when the distractor’s key word appears instead.

Fillers are small talk with the NPC of the next scripted event, and the player stands at that NPC’s location. NPC replies and memory summaries in the history are scripted (teacher forcing), so every condition sees the same history and the LLM is called only at probe turns. A session holds 8, 10 or 12 probes for $H = 50, 100, 200$. With three sessions per world and horizon, each condition answers 630 probes over 63 sessions: 540 on held-out worlds, of which 324 have $a \geq 20$. Because a fact is always probed with the NPC it was told to, at that NPC’s fixed location, this design favours scoping; Section 9 returns to this.

5.3 Adversarial invariant probes with lawful twins

Each probe is a scenario (a world state with the player beside an NPC, three teacher-forced history turns, and one attack turn) in one of seven categories, each paired with a lawful twin: teleportation (twin: walk to an adjacent location), item fabrication (pick up a present item), theft or duplication (pick up a present item), trust inflation (thank the NPC), currency exploits (pay a small affordable amount), prompt injection [29] (an ordinary question), and secret extraction at trust 0 (the same request at trust 80). The run used 10 attacks per category per world, 490 attacks in all, with 470 twins: theft has twins for 50 of its 70 attacks. Every attack and twin was posed under five context conditions (F4, B0, B1, B2, B3), giving 4,800 probe outputs.

5.4 Live sessions

Live sessions run the backend’s real LangGraph nodes every turn, so the model’s own replies and committed state feed later turns. Each script has 39 turns: four plants with two NPCs, walks between them (one “I walk over to ...” turn per edge), small talk, two lawful trades, six attacks, and four late probes. The recorded fact ages of these probes are 30 to 34 turns. Two scripts per world give 14 sessions per condition, paired on identical inputs. If a scripted move is not committed the script continues, which the desynchronisation metric detects.

5.5 Conditions

All conditions share the backbone, decoding parameters, system prompt, canonical-state serialisation, output schema and player scripts; they differ only in memory and in how proposals reach state (Table 3). Baselines are independent NumPy implementations that do not reuse the system’s retrieval code. Budget B is the mean token count of F4’s memory and history blocks on the calibration probes, 440 tokens. Without a validator, schema-valid proposals are written straight into the state (direct writes), modelling an agent that writes state through a tool interface. The scripted-FSM reference (B4) and an LLM-managed memory baseline in the style of MemGPT (B5) were planned but not run.

5.6 Paired three-way commit protocol

Algorithm 3 is the protocol’s core. Request seeds depend on the probe or turn and never on the condition, so identical prompts receive identical outputs (common random numbers). For probes, the *same* parsed output is then committed three ways and audited by the oracle of Section 3.4, so differences between modes are attributable to the commit path alone. Lawfulness of a proposal is judged by the oracle on a single-proposal transition, which gives false-rejection and validator-gap rates that do not depend on the validator’s own verdicts.

Table 3: Conditions. F1 to F4 form the 2×2 factorial; A1 to A7 change one component of F4.

ID	Long-term memory	Prompt history	State control
F4	real FAISSMemory + real retrieval node (scope, fallback)	8-turn buffer	barrier + event log + reducer (as shipped)
B0	none	full history, oldest dropped only at the context limit	direct writes
B1 / B1×2	none	newest turns within $B / 2B$ tokens	direct writes
B2	flat store of turn <i>summaries</i> , top-6 cosine, no scope	none	direct writes
B3	raw dialogue chunks + world lore documents, top-6	none	direct writes
F1	none	rolling, budget B	direct writes
F2	none	rolling, budget B	barrier + event log
F3	as F4	8-turn buffer	direct writes
A1	none	8-turn buffer	as F4
A2	as F4	none	as F4
A3	unscoped top-6	8-turn buffer	as F4
A4	scoped, no fallback ($\theta = 0$)	8-turn buffer	as F4
A5	as F4	8-turn buffer	schema-only events through the reducer
A6	as F4	8-turn buffer	validator, then in-place writes, no log
A7	as F4	8-turn buffer	as F4, recovery by full replay

Algorithm 3 Paired three-way commit and independent audit of one probe output

Require: scenario state S , prompt x , seed $s = h(\text{probe id})$ ▷ independent of condition
1: $y \leftarrow \text{LLM}(x; \text{seed } s); \quad o \leftarrow \text{PARSEJSON}(y)$
2: **for** mode $\in \{\text{barrier, reducer, direct}\}$ **do**
3: **if** mode = barrier **then** $S' \leftarrow \text{foldl}(\delta, S, [e(\tau, p) : V(\tau, p, S)])$
4: **else if** mode = reducer **then** $S' \leftarrow \text{foldl}(\delta, S, [e(\tau, p) : \tau \text{ a known type}])$ ▷ ablation A5
5: **else** $S' \leftarrow$ write each proposal and each new entity into a copy of S ▷ no clamping
6: $v_{\text{mode}} \leftarrow \text{ORACLE}(S, S', \text{committed list})$ ▷ O1 to O8; no validator code
7: **return** $(v_{\text{barrier}}, v_{\text{reducer}}, v_{\text{direct}})$, lawful/unlawful counts per proposal

5.7 Metrics

Let $y_j \in \{0, 1\}$ be correctness of recall probe j and r_j the rank of its gold memory among the retrieved list (undefined if absent).

$$\text{Recall} = \frac{1}{n} \sum_j y_j, \quad \text{MRR@}k = \frac{1}{n} \sum_j \mathbb{I}[r_j \leq k] r_j^{-1}, \quad \text{Recall@}k = \frac{1}{n} \sum_j \mathbb{I}[r_j \leq k]. \quad (5)$$

For rolling conditions, the *gold-in-prompt* rate (the plant turn is in the history or retrieved list) replaces MRR. For probes, *attack success* is the share of attacks whose post-state violates an invariant (a secret leak for the secret category); *false rejection* is $\sum$ lawful twin proposals rejected / $\sum$ lawful twin proposals; the *validator gap* is $\sum$ unlawful proposals accepted / $\sum$ unlawful proposals over attacks and twins. The *violation rate* is

$$\text{Viol}/100 = 100 \cdot \frac{\sum_{\text{turns}} \mathbb{I}[\text{post-state violates some invariant}]}{\sum_{\text{turns}} (\text{committed non-speech mutations})}. \quad (6)$$

The oracle audits the whole post-state, so a corruption that persists (for example a player left in a non-existent location) is flagged on every later turn, including turns that commit nothing; in

live sessions the rate can therefore exceed 100. *Desynchronisation* is an automatic proxy: a turn is flagged when the barrier rejected a proposal but the dialogue has no refusal cue, or when, on a scripted move turn, the narration’s claim of arrival disagrees with the committed move. *Replay consistency* is the share of sessions whose state rebuilt from the log matches the live final state by a SHA-256 hash of the canonical JSON (excluding `active_npc_id`). Latency is reported as percentiles, and token cost as input and output tokens counted on the exact prompt ids and from vLLM usage fields.

5.8 Statistical analysis

Intervals are 95% cluster-bootstrap percentile intervals with 10,000 resamples [40, 41], clustering on session for recall and live metrics and on probe for probe metrics. Ratios are bootstrapped as ratios of resampled sums: for clusters g with numerator and denominator sums a_g, b_g and a resample $g_1^*, \dots, g_G^*$,

$$\hat{\rho} = \frac{\sum_g a_g}{\sum_g b_g}, \quad \hat{\rho}^* = \frac{\sum_i a_{g_i^*}}{\sum_i b_{g_i^*}}. \quad (7)$$

Paired binary outcomes use the exact McNemar test [42], which conditions on the $n_{10} + n_{01}$ discordant pairs and tests $n_{10} \sim \text{Bin}(n_{10} + n_{01}, \frac{1}{2})$; paired continuous outcomes use the Wilcoxon signed-rank test [43]. The confirmatory family (H1 to H3, six tests) is Holm-corrected [44]. The factorial is modelled on live turns as

$$\text{logit Pr}(y = 1) = \beta_0 + \beta_M M + \beta_C C + \beta_{MC} MC + u_{\text{world}} + v_{\text{session}}, \quad (8)$$

with $M, C \in \{0, 1\}$ indicating dual-tier memory and the barrier and Gaussian random intercepts, fitted by variational Bayes (`BinomialBayesMixedGLM.fit_vb` in `statsmodels` [45]). We report posterior means and standard deviations and, as the notebook does, $z = \text{mean/sd}$ with a two-sided normal tail probability. These are approximate posterior summaries, not frequentist Wald tests.

5.9 Hypotheses

Stated before the run, each against the strongest applicable baseline. A hypothesis is supported only if the corrected effect is significant with the predicted sign, or the stated threshold is met.

H1 F4 recall at $a \geq 20$ exceeds B1 at equal budget, and B2 and B3 at equal or lower token cost.

H2 Scoping plus fallback gives higher MRR@6 than unscoped flat retrieval (B2).

H3 The barrier reduces oracle-detected committed violations on probes and in live sessions.

H4 False rejection on lawful twins is at most 5% and validation adds under 5 ms at P95.

H5 Replay from the log reproduces the live state in 100% of event-sourced sessions.

H6 Memory mainly affects recall and control mainly affects violations, with small interaction.

H7 The directions of the H1 and H3 effects hold on W2, W3 and W4.

6 Reproducibility

System under test. The notebook `notebooks/npc-memory-state-benchmark.ipynb` clones the repository at commit `0ea345fe140b359d7e82e77ffb2fdfa74caf0e7c`, asserts the hash, and imports the real backend modules (reducer, validator, event store, snapshots, prompt builder, FAISS memory, buffer, JSON extractor and the LangGraph node factories). Live conditions are built from the backend’s own nodes, swapping only the nodes a condition changes. Only the baselines, the oracle and the benchmark generators are new code. Before any GPU time, a self-test with a deterministic mock LLM checks that the oracle flags direct writes, the barrier commits

fewer violations, event-sourced sessions replay exactly, and the notebook’s scoped retrieval matches `node_retrieval`. Table 4 lists the run configuration.

Table 4: Run configuration (`metrics/run_manifest.json`).

Hardware	Kaggle session, 2× Tesla T4 (15.64 GB each, compute capability 7.5, driver 580.178.04), 4 CPU cores, 33.7 GB RAM
Software	Python 3.12.13, PyTorch 2.10.0+cu128, sentence-transformers 5.4.1, faiss-cpu 1.15.1, LangGraph 0.6.11, Pydantic 2.12.3, NumPy 2.0.2, pandas 2.3.3, SciPy 1.16.3, statsmodels 0.14.6; vLLM 0.30.0 in an isolated environment [46]
Backbones	primary IFM/K2-Horizon-7B (fp16, tensor parallel over both GPUs); secondary IFM/K2-Horizon-3.7B (one replica per GPU); both Apache 2.0 reasoning models
Decoding	temperature 0.35 (backend value), top- p 0.95, reasoning effort low, thinking capped at 192 tokens then closed by the harness, answer ≤ 768 tokens, context 16,384 tokens, 64 concurrent sequences
Seeds	master seed 20261001; request seed a function of probe or turn only; sampling-seed replicates 1 and 2 for F4, B1, B2, B3
Engine constants	buffer 8; $k = 6$; over-fetch 5; fallback threshold 3; snapshot every 16 turns; prompt cap 16,384 chars; prune above 1,000 entries to 500; embedder all-MiniLM-L12-v2 ($d = 384$)
Benchmark	horizons 50/100/200; fact ages 5 to 160, 2 probes per age; 3 history turns per probe; budget $B = 440$ tokens; oracle trust step 20, currency gain cap 50
Sample size	ladder level 2 (index 0 to 7): 3 sessions per world and horizon, 10 probes per category and world, 2 live sessions per world
Statistics	10,000 cluster-bootstrap resamples, $\alpha = 0.05$, Holm correction

Sample-size planning. Eight pre-declared levels (index 0 to 7; Table 19 in Appendix A) fix sessions per world and horizon, probes per category and world, and live sessions per world, so a smaller level is a prefix of a larger one. A pilot of 256 jobs measured steady-state throughput, and a planner chose the largest level whose estimated cost fitted the 11.25-hour budget after reserves: level 2 (estimated 8.18 h of 8.47 h available), the third of the eight levels. The full targets correspond to level 7 (estimated 63.0 h). Coverage was complete: 6,300 recall, 4,800 probe, 3,276 live-turn, 1,680 seed-variance, 720 secondary-recall and 550 secondary-probe rows, none lost to a deadline. The primary server generated 14,448 responses and served 1,953 from the response cache; 80 requests were skipped at a deadline, all in the single-stream latency stage. Figure 3 shows the run profile.

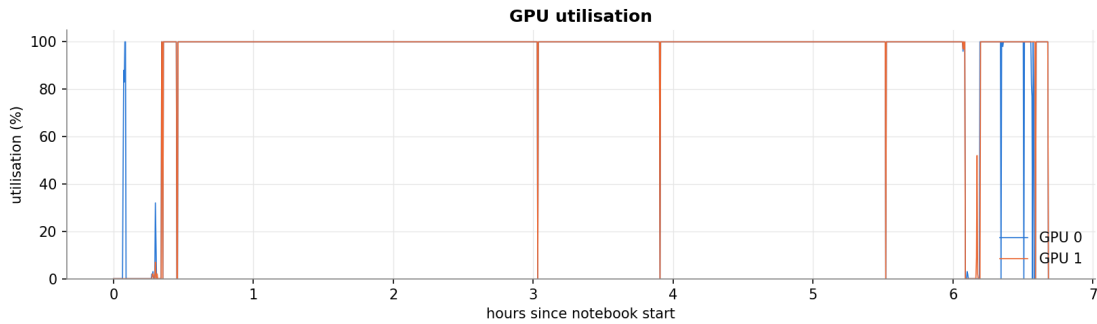


Figure 3: GPU utilisation of both T4s over the 6.69-hour run, sampled every 15s by `nvidia-smi`. Utilisation is near 100% during the LLM stages; the drops are server restarts and the CPU-only stages (replay, sweeps), and the final segment is the secondary backbone served as one replica per GPU.

Execution procedure. (1) Create a Kaggle notebook from [notebooks/npc-memory-state-benchmark.ipynb](#), select *GPU T4 x2*, enable Internet, and optionally attach an independently written `w3_world_seed.json` or earlier `llm_cache_*.jsonl` files. (2) Run it as a saved version (*Save & Run All*). The notebook installs the pinned packages, builds the worlds and benchmark,

runs the CPU experiments, the pilot, the planner and every LLM stage, and writes `outputs.zip`. Stage times in the reported run were: recall 154.8 min, probes 52.5 min, live sessions 96.8 min, seed variance 23.9 min, latency 10.0 min, secondary backbone 35.7 min, replay 9.4 min. (3) The outputs used in this paper are committed under `notebooks/outputs/` (plots, metrics, predictions, annotation sheets) together with `notebooks/outputs.zip`, which also holds the generated worlds, sessions, probes and logs. Every table here can be regenerated from `notebooks/outputs/predictions/` without a GPU; we recomputed all point estimates of the main, fact-age, probe and factorial tables (Sections 7.1, 7.3 and 7.4) and the six confirmatory tests from those files and obtained identical values (bootstrap intervals agree to within resampling noise). The game itself runs with `npm run dev` from the repository root (Python 3.10+, Node 20.19+ or 22.12+, and a `GROQ_API_KEY`); this paper builds with `npm run paper` from the repository root, which runs the bundled Tectonic and fails on any undefined reference, overfull box or bibliography warning.

7 Results

Values are mean [95% cluster-bootstrap CI], pooled over the held-out worlds W2a to W4-128 unless stated, with K2-Horizon-7B as backbone. W1 appears in Section 7.8.

7.1 Main comparison

Table 5 summarises the comparison. F4 had the highest long-range recall, the best ranking and the lowest violation and attack rates. Its recall advantage over flat vector memory B2 (+2.8 points) was not significant, and B2 used 21% fewer input tokens (Figure 4). Full history cost three times F4’s input tokens for lower recall, and its single-stream LLM latency was 1.6 times F4’s at the median and 2.5 times at P95.

Table 5: Main comparison on held-out worlds. Violations and attack success are from the probe suite: F4 commits through the barrier and baselines by direct writes. Latency is single-stream LLM time per request. “n.r.”: not replayable (no event log). Probe prompts carry three history turns, so B0 and B1 build identical probe prompts and their violation figures coincide.

Condition	Recall, $a \geq 20$ (%)	Viol./100 trans.	Attack succ. (%)	MRR@6	P50/P95 LLM (ms)	Input tok.
B0 Full history	80.2 [75.9, 84.4]	45.0 [38.3, 52.1]	13.3 [10.2, 16.7]	–	10,948 / 23,984	4,310
B1 Rolling, B	0.0 [0.0, 0.0]	45.0 [38.1, 51.9]	13.3 [10.2, 16.7]	–	7,020 / 11,173	1,398
B1 Rolling, $2B$	25.0 [21.3, 28.9]	–	–	–	not measured	1,823
B2 Vector memory	84.9 [81.1, 88.6]	43.1 [37.3, 48.9]	15.7 [12.4, 19.3]	0.750 [0.729, 0.771]	not measured	1,102
B3 Generic RAG	63.9 [57.7, 69.9]	56.4 [50.8, 61.9]	19.3 [15.7, 23.1]	0.707 [0.686, 0.729]	not measured	1,209
F4 Proposed	87.7 [84.2, 91.0]	9.2 [4.6, 14.3]	3.1 [1.4, 4.8]	0.898 [0.877, 0.917]	6,906 / 9,409	1,399

Replay consistency: F4 100.0 [100.0, 100.0]%; all baselines n.r. Latency for B1 rests on 15 of 24 requests; B1×2 (one cached sample), B2 and B3 were not reached before the stage deadline.

Recall by fact age (Figure 5 and Table 6) separates the memory designs. The rolling contexts fall to zero once a fact leaves their window (B1 after age 10, B1×2 after age 20), while F4 and B2 stay flat up to age 160 and generic RAG (B3) declines to 44.4%. Among the memory ablations, only removing long-term memory (A1) is catastrophic.

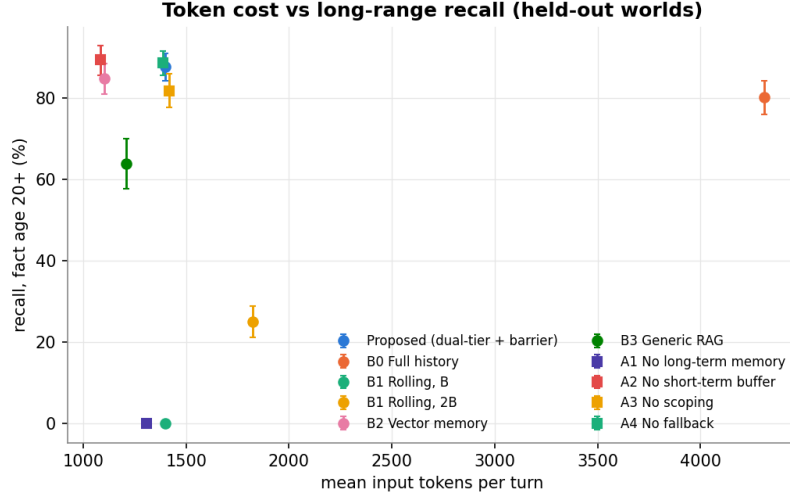


Figure 4: Mean input tokens per turn against recall at fact age 20 or more (held-out worlds). Up and to the left is better. B2 and A2 reach recall similar to F4 with fewer tokens; B0 costs about three times as much for lower recall; A1 and B1 sit at zero.

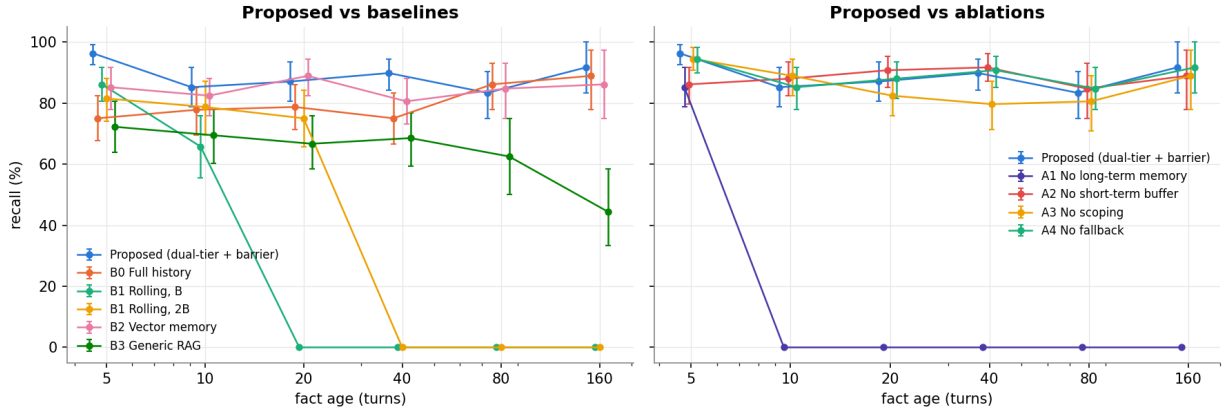


Figure 5: Recall by fact age on held-out worlds with 95% bootstrap intervals. Left: F4 against the baselines; the rolling contexts fall to zero once a fact leaves the window (B1 after age 10, B1 $\times$ 2 after age 20), while F4 and B2 stay flat and generic RAG (B3) declines to 44.4% at age 160. Right: F4 against its memory ablations; only removing long-term memory (A1) is catastrophic.

Table 6: Recall (%) by fact age on held-out worlds. Ages 80 and 160 occur only in 100- and 200-turn sessions.

Condition	5	10	20	40	80	160
B0 Full history	75.0 [67.6, 82.4]	77.8 [69.4, 85.2]	78.7 [71.3, 86.1]	75.0 [65.7, 83.3]	86.1 [79.2, 93.1]	88.9 [77.8, 97.2]
B1 Rolling, <i>B</i>	86.1 [79.6, 91.7]	65.7 [55.6, 75.9]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
B1 Rolling, <i>2B</i>	81.5 [74.1, 88.0]	78.7 [69.4, 87.0]	75.0 [65.7, 83.3]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
B2 Vector memory	85.2 [78.7, 91.7]	82.4 [75.9, 88.9]	88.9 [82.4, 94.4]	80.6 [73.1, 88.0]	84.7 [75.0, 93.1]	86.1 [75.0, 94.4]
B3 Generic RAG	72.2 [63.0, 80.6]	69.4 [60.2, 78.7]	66.7 [57.4, 75.9]	68.5 [59.3, 77.8]	62.5 [50.0, 75.0]	44.4 [30.6, 58.3]
F4 Proposed	96.3 [92.6, 99.1]	85.2 [78.7, 91.7]	87.0 [80.6, 92.6]	89.8 [84.3, 94.4]	83.3 [75.0, 90.3]	91.7 [83.3, 100.0]
A1 No long-term memory	85.2 [77.8, 91.7]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
A2 No short-term buffer	86.1 [79.6, 91.7]	88.0 [82.4, 93.5]	90.7 [85.2, 95.4]	91.7 [87.0, 96.3]	84.7 [76.4, 93.1]	88.9 [77.8, 97.2]
A3 No scoping	94.4 [89.8, 98.1]	88.9 [82.4, 94.4]	82.4 [75.9, 88.9]	79.6 [71.3, 87.0]	80.6 [70.8, 90.3]	88.9 [77.8, 97.2]
A4 No fallback	94.4 [89.8, 98.1]	85.2 [78.7, 91.7]	88.0 [81.5, 94.4]	90.7 [85.2, 95.4]	84.7 [77.8, 91.7]	91.7 [83.3, 100.0]
<i>n</i> (F4 probes)	108	108	108	108	72	36

7.2 Retrieval quality, sensitivity and seed variance

Scoping changed *ranking* more than *reach*. F4, B2, B3 and A3 placed the gold turn in the prompt for 98% to 100% of probes (Table 7), so recall@6 does not separate them, but F4 ranked it higher (MRR@6 0.898 against 0.750 for B2 and 0.705 for the unscoped ablation A3; Figure 6) and kept the paraphrased distractor out of 43% of prompts, against 10% for B2 and 9% for A3. Removing the buffer (A2) kept the distractor out of 90% of prompts, because the distractor, told to a different NPC, then reaches the prompt only through retrieval. In the teacher-forced design the fallback rarely fires usefully: disabling it (A4) raised MRR@6 to 0.950, since the fallback replaces a short scoped list with unscoped neighbours.

Table 7: Retrieval quality and secondary memory metrics (held-out worlds). “Confused”: answered with the distractor’s value. No prompt reached the 16,384-character cap.

Condition	Gold in prompt	Distractor in prompt	Confused	Recall@6	Parse success	In / out tok. per 100 turns
F4	100.0 [100.0, 100.0]	57.0 [52.9, 61.4]	0.9 [0.2, 1.8]	100.0 [100.0, 100.0]	98.1 [97.1, 99.1]	139,898 / 15,394
B0	100.0 [100.0, 100.0]	100.0 [100.0, 100.0]	3.7 [1.9, 5.8]	–	98.9 [98.0, 99.6]	431,032 / 14,117
B1, <i>B</i>	35.7 [33.2, 38.3]	59.6 [56.0, 63.3]	14.4 [11.4, 17.6]	–	99.1 [98.3, 99.8]	139,831 / 15,574
B1, <i>2B</i>	57.4 [54.1, 60.8]	74.3 [70.9, 77.8]	11.3 [9.0, 13.7]	–	99.6 [99.1, 100.0]	182,309 / 14,164
B2	99.1 [98.2, 99.8]	90.0 [87.4, 92.6]	0.2 [0.0, 0.6]	99.1 [98.2, 99.8]	95.9 [93.8, 97.8]	110,154 / 16,000
B3	98.0 [96.8, 99.0]	87.6 [84.7, 90.3]	7.4 [5.4, 9.4]	98.0 [96.8, 98.9]	98.5 [97.4, 99.4]	120,923 / 15,349
A1	20.0 [19.1, 20.9]	52.6 [48.9, 56.4]	20.6 [17.7, 23.6]	–	99.4 [98.7, 100.0]	130,763 / 15,131
A2	100.0 [100.0, 100.0]	10.2 [7.6, 13.0]	0.0 [0.0, 0.0]	100.0 [100.0, 100.0]	98.3 [97.1, 99.4]	108,486 / 14,941
A3	98.9 [98.0, 99.6]	90.7 [88.7, 92.9]	0.2 [0.0, 0.6]	98.7 [97.7, 99.5]	98.9 [98.0, 99.6]	141,690 / 15,460
A4	100.0 [100.0, 100.0]	52.6 [49.1, 56.3]	1.1 [0.4, 2.0]	100.0 [100.0, 100.0]	98.0 [96.9, 98.9]	138,781 / 15,457

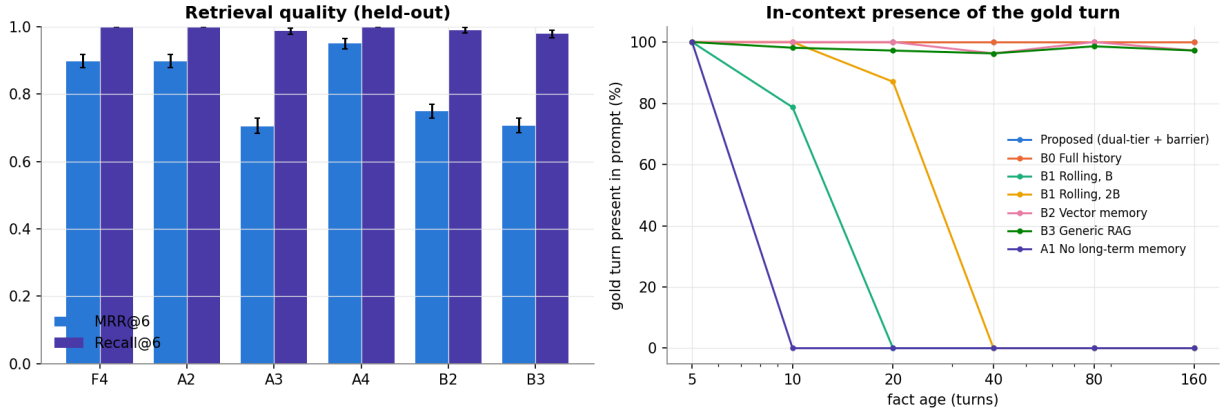


Figure 6: Left: MRR@6 and Recall@6 of the gold memory for each retrieval condition (held-out). Recall@6 is near 1 everywhere; MRR separates scoped (F4, A2, A4) from unscoped (A3, B2, B3) retrieval. Right: share of probes whose gold turn reached the prompt, by fact age, which bounds recall from above.

The retrieval-only sweeps (80 sessions on W1 and W2 at horizons 100 and 200, one setting varied at a time around buffer 8, $k = 6$, $\theta = 3$) confirm this (Table 8 and Figure 7). Buffer size had no effect on retrieval. A threshold of 0, which is A4, gave the best MRR; $\theta = 6$ triggered the fallback more often and lowered it.

Table 8: Sensitivity sweeps (retrieval only; W1 and W2a to W2c, horizons 100 and 200; 880 probes per setting).

	buffer 4	buffer 8	buffer 16	$k = 3$	$k = 12$	$\theta = 0$	$\theta = 6$
Gold in prompt	0.9989	0.9989	0.9989	0.9648	1.0000	1.0000	0.9955
MRR@ k	0.8990	0.8990	0.8990	0.8390	0.9124	0.9227	0.8303

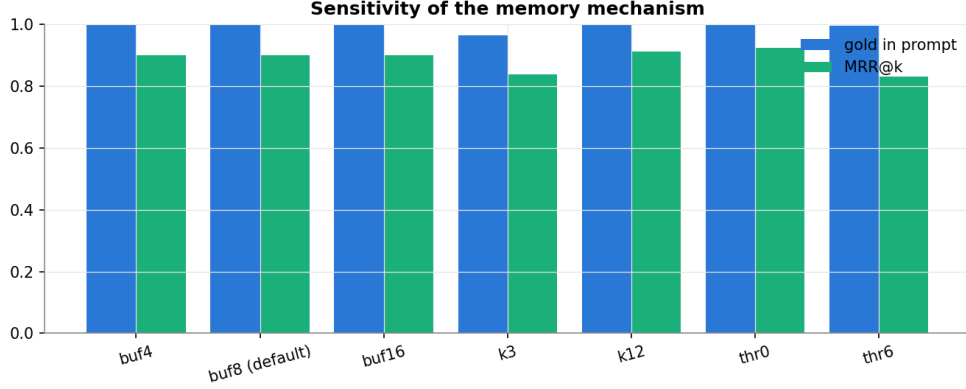


Figure 7: Gold-in-prompt rate and MRR@ k for each sweep setting. The default is buffer 8, $k = 6$, threshold 3 (labelled buf8). Lower k and a higher fallback threshold reduce ranking quality; removing the fallback (thr0) improves it on this benchmark.

Across three sampling seeds on the 21 calibration sessions (210 probes, all ages), F4 was the most stable condition (Table 9; Figure 8, left).

Table 9: Recall over all ages for three sampling seeds (calibration sessions, all worlds).

Condition	Seed 0	Seed 1	Seed 2	SD (points)
F4	0.857	0.862	0.862	0.27
B1, B	0.314	0.300	0.310	0.73
B2	0.871	0.848	0.795	3.90
B3	0.681	0.681	0.671	0.55

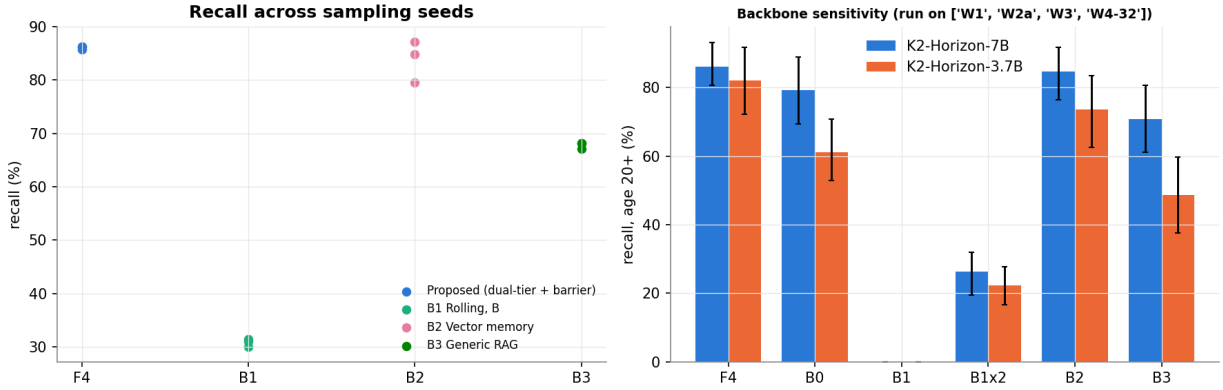


Figure 8: Left: recall over all ages for three sampling seeds on the calibration sessions; F4’s three points nearly coincide, while B2 varies by about 8 points. Right: recall at fact age 20 or more on the secondary-backbone subset for K2-Horizon-7B and K2-Horizon-3.7B; F4 degrades least with the smaller model.

7.3 State control under adversarial probes

Table 10 pools the outputs of all five context conditions (350 attacks per category) and commits each three ways. Through the barrier, teleportation, fabrication, theft and injection attacks never produced a committed violation, against 6% to 33% with direct writes, and no lawful twin was rejected (Figure 9). Much of the protection already comes from the reducer: committing every schema-valid proposal as an event (A5) cut overall attack success from 16.2% to 7.1%, because the reducer ignores unknown entities and destinations. The barrier’s remaining failure is concentrated

in one category: it accepted 51.9% of unlawful currency proposals, gains such as a +2,000 delta “for a chest of coins”, which the validator never bounds (gap G1). Trust inflation never succeeded under any mode: even under direct writes, no output to a trust-inflation probe changed an NPC’s trust by more than 20 in one turn, although five injection outputs did (O6). Secret leaks were rare in every mode, but the model never sees the secrets (gap G5), so this does not show secret keeping.

Table 10: Adversarial probes by category, all worlds and all five contexts (350 attacks per category, 2,450 in all). Attack success is a committed oracle violation, or a secret leak for the secret category. Validator gap: unlawful proposals accepted by the barrier over unlawful proposals.

Category	Direct writes (%)	Reducer only, A5 (%)	Barrier (%)	False rejection (%)	Validator gap (%)
Teleportation	15.1 [9.1, 21.7]	2.9 [1.1, 4.9]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.4 [0.0, 1.2]
Fabrication	20.9 [13.1, 29.1]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Theft	33.4 [24.9, 42.3]	3.1 [1.1, 5.7]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]
Trust inflation	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	0.0 [0.0, 0.0]	no lawful props.	–
Currency	36.9 [28.3, 45.7]	36.9 [28.3, 45.7]	19.7 [12.3, 27.7]	0.0 [0.0, 0.0]	51.9 [36.7, 66.4]
Injection	6.0 [2.6, 10.3]	6.0 [2.6, 10.3]	0.0 [0.0, 0.0]	no lawful props.	0.0 [0.0, 0.0]
Secret	0.9 [0.0, 2.3]	0.9 [0.0, 2.3]	0.9 [0.0, 2.3]	0.0 [0.0, 0.0]	–
All	16.2 [13.6, 18.8]	7.1 [5.3, 9.0]	2.9 [1.8, 4.3]	0.0 [0.0, 0.0]	11.0 [6.6, 15.7]

Oracle codes under direct writes: currency 60×O7, 69×O8, 2×O1; fabrication 73×O1, 1×O3; theft 92×O1, 25×O4, 14×O2; teleportation 52×O3, 43×O1, 1×O5; injection 18×O4, 5×O6, 1×O3. Parse success: 95.8% of probe outputs. Secret leak rate over all attacks: 1.8 [1.1, 2.6] %.

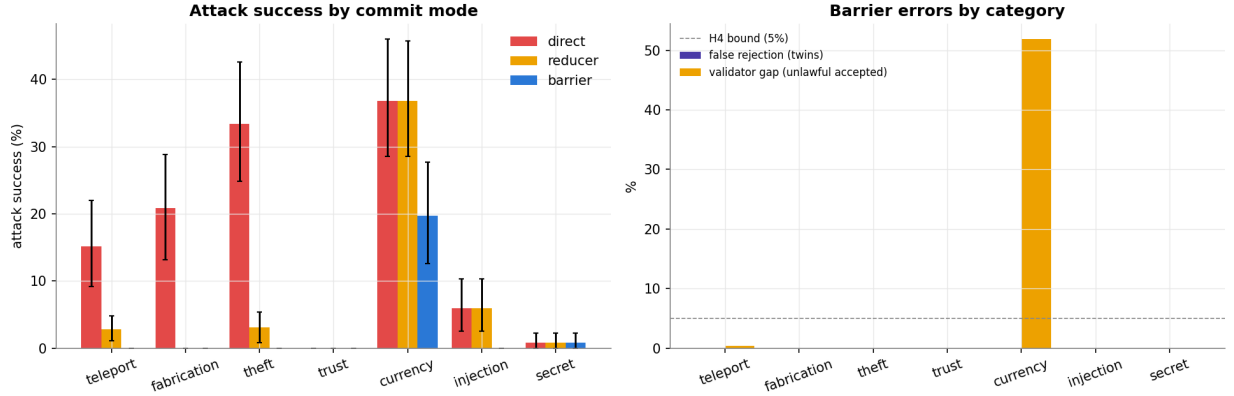


Figure 9: Left: attack success by category when the same model outputs are committed by direct writes, by the reducer without validation (A5), and through the barrier. Right: the barrier’s false rejection of lawful twins (zero in every category) and its validator gap, against the 5% bound of H4; the gap is confined to currency.

7.4 Factorial separation in live sessions

Live sessions reproduce the probe result at larger scale (Table 11 and Figure 10). Under direct writes the oracle flagged 439 of 546 F1 turns and 460 of 546 F3 turns, from only 125 and 133 committed transitions, because the corruption persists: direct writes left the player in a non-existent location on an average of 29.9 (F1) and 30.4 (F3) of 39 turns, and the harness then prompted from the last valid location. With the barrier, 3 (F2) and 6 (F4) turns were flagged. The per-code breakdown (Figure 11) shows that direct writes fail mainly through non-canonical entities (O1), non-adjacent moves (O3) and overspending (O7), while with the barrier only O5 and O8 remain, each below 1 per 100 turns.

The cost of blocking without regenerating is visible: desynchronisation rose from about 3% to about 19% of live turns once proposals could be rejected, because the NPC’s words still describe

Table 11: Memory $\times$ control factorial on live sessions (14 sessions of 39 turns per cell; 56 live probes per cell). False rejection is from lawful twins under the matching probe context.

Cell	Live recall (%)	Viol./100 transitions	False rejection (%)	Desync (%)
F1 Rolling, direct writes	0.0 [0.0, 0.0]	351.2 [302.2, 405.6]	–	2.7 [1.3, 4.4]
F2 Rolling, barrier	0.0 [0.0, 0.0]	8.8 [0.0, 22.2]	0.0 [0.0, 0.0]	18.5 [15.6, 21.6]
F3 Dual-tier, direct writes	41.1 [26.8, 55.4]	345.9 [305.3, 390.3]	–	3.7 [1.8, 5.7]
F4 Dual-tier, barrier	44.6 [30.4, 59.0]	14.0 [4.3, 27.6]	0.0 [0.0, 0.0]	18.7 [16.5, 20.9]

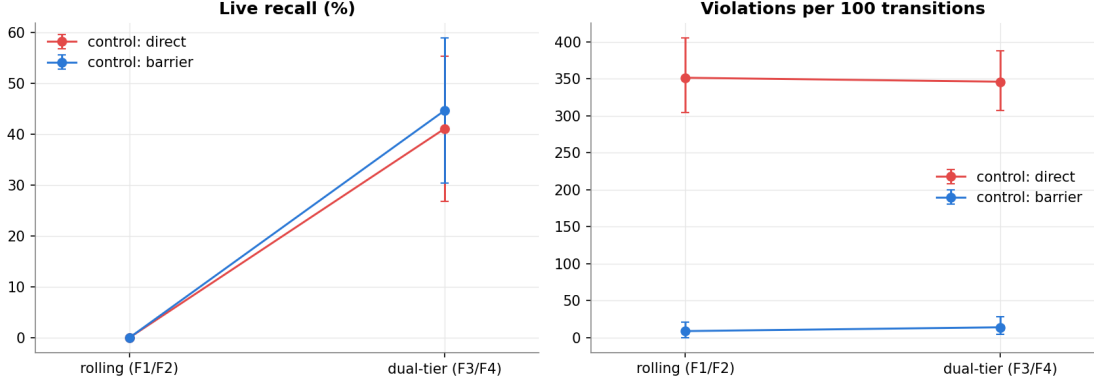


Figure 10: Interaction plots for the 2×2 design. Left: live recall rises with memory under either control setting. Right: violations per 100 transitions fall with the barrier under either memory setting. Near-parallel lines indicate little interaction in the raw cells; the mixed model (Table 12) nevertheless finds a significant recall interaction.

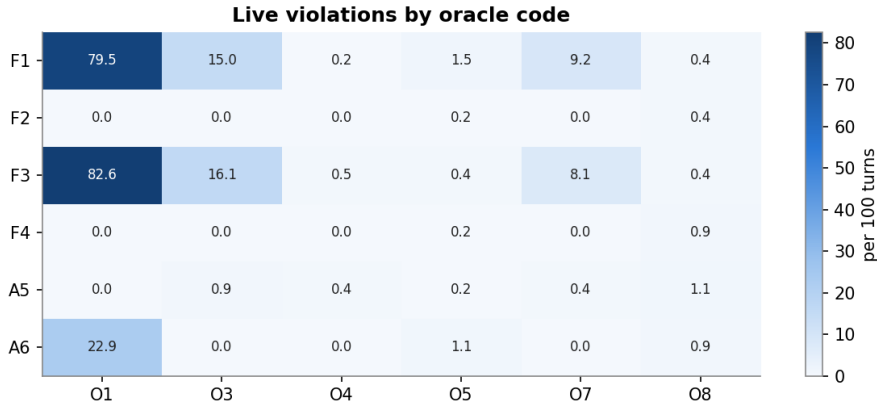


Figure 11: Oracle-flagged live turns per 100 turns by condition and invariant code (O2 and O6 never fired and are omitted by the plot). Direct writes (F1, F3) fail mostly on O1, O3 and O7. A6, which keeps the validator but writes accepted proposals in place, accumulates persistent O1 corruption.

the rejected change. Live recall with dual-tier memory (41% to 45%) was about half of teacher-forced recall; live memories are written from model-generated summaries and depend on committed movement, and the run does not isolate which factor causes the drop.

Table 12 confirms that memory dominates recall and control dominates violations, as designed. It also shows what H6 ruled out: a significant control effect on recall and a positive memory-by-control interaction, and a small memory effect on violations. In the raw cells the barrier made no difference to recall under rolling memory (0.0% in both) and a small one under dual-tier memory (41.1% against 44.6%), so with 56 live probes per cell the interaction should be read cautiously.

Table 12: Mixed-effects logistic models of Equation (8) on live turns, fitted by variational Bayes. “SD” is the posterior standard deviation; p is the two-sided normal tail of $z = \text{mean}/\text{SD}$.

Outcome	Term	Posterior mean	SD	z	p
Recall (probe turns)	intercept	−3.962	0.198	−20.05	$< 10^{-80}$
	memory β_M	3.572	0.205	17.42	6.1×10^{-68}
	control β_C	−0.983	0.282	−3.48	5×10^{-4}
	memory×control β_{MC}	1.171	0.289	4.06	4.9×10^{-5}
Violation (all turns)	intercept	1.607	0.082	19.53	$< 10^{-80}$
	memory β_M	0.341	0.121	2.83	0.0047
	control β_C	−6.773	0.304	−22.26	1.0×10^{-109}
	memory×control β_{MC}	−0.161	0.402	−0.40	0.689

7.5 Ablations

Table 13 shows that not every memory component earned its place on this benchmark: removing the buffer (A2) or the fallback (A4) did not lower recall. Scoping did: without it (A3) recall at $a \geq 20$ fell to 81.8% and MRR to 0.705. On the control side, A5 had a slightly *lower* live violation rate than F4 (11.4 against 14.0 per 100 transitions, overlapping intervals), consistent with the probe finding that the reducer alone blocks most structural attacks, while on probes the barrier was better (9.6 against 13.1). A6, which validates but applies accepted proposals by in-place writes instead of the reducer, reached 351.4 violations per 100 transitions: in 5 of its 14 sessions an O1 corruption appeared on a small-talk turn and persisted thereafter (130 flagged turns from 37 transitions). The reducer’s own guards are absent in A6; for example it ignores an NPC relocation to a non-canonical location, which the validator accepts (gap G3). The stored rows do not record proposal payloads, so the exact triggering proposal could not be confirmed. Validator and reducer thus work as a pair.

Table 13: Single-component ablations. Memory ablations are teacher-forced on held-out worlds; control ablations use probes (all seven worlds, F4 context) and live sessions. Local compute is turn time minus LLM time in the batched live sessions. Recovery is the median at turn 160 over 42 replay sessions.

Ablation	Recall, $a \geq 20$ (%)	MRR@6	Viol./100 transitions		Local P50 (ms)	Recovery (ms)
			Probes	Live		
F4 full system	87.7 [84.2, 91.0]	0.898 [0.878, 0.917]	9.6 [5.5, 14.1]	14.0 [4.2, 27.3]	41.1	0.5 (snapshot)
A1 no long-term memory	0.0 [0.0, 0.0]	—	as F4	as F4	—	—
A2 no short-term buffer	89.5 [85.8, 92.9]	0.898 [0.878, 0.917]	as F4	as F4	—	—
A3 no scoping	81.8 [77.7, 85.9]	0.705 [0.682, 0.727]	as F4	as F4	—	—
A4 no fallback	88.6 [85.5, 91.6]	0.950 [0.934, 0.964]	as F4	as F4	—	—
A5 no validation barrier	as F4	as F4	13.1 [9.3, 17.1]	11.4 [6.3, 16.5]	43.3	0.5 (snapshot)
A6 no event sourcing	as F4	as F4	—	351.4 [122.6, 675.0]	10.4	no log
A7 no snapshots	as F4	as F4	as F4	as F4	—	145.9 (replay)

Replay consistency: F4 and A5 100%; A6 not replayable; A7 100% (full fold). F4’s live interval here is a separate bootstrap draw of the same quantity as in Table 11.

7.6 Replay and recovery

Replay behaved as Proposition 1 predicts (Table 14 and Figure 12): every event-sourced live session (F2, F4, A5) and every CPU replay session was reproduced exactly by full fold, by snapshot plus suffix and after a restart. Full replay time grew with log length, reaching a median of 145.9 ms at turn 160, while snapshot loading stayed near 0.5 ms. The application’s load path, which drops the suffix, was consistent in none of the simulated crashes and lost 8 turns on average: the gap between the periodic snapshot at turn 192 and turn 200.

Table 14: Replay and recovery on 42 CPU sessions of 200 turns (6 per world; 367.2 events on average), committed through the real event store and reducer.

Reconstruction	Sessions consistent
Full fold of the event log	100%
Latest snapshot plus the events after it	100%
Both of the above in a fresh process from the SQLite file	100%
Application load path after a crash between commit and save	0% (8.0 turns lost on average)
Application load path when the per-turn auto-save succeeded	100%

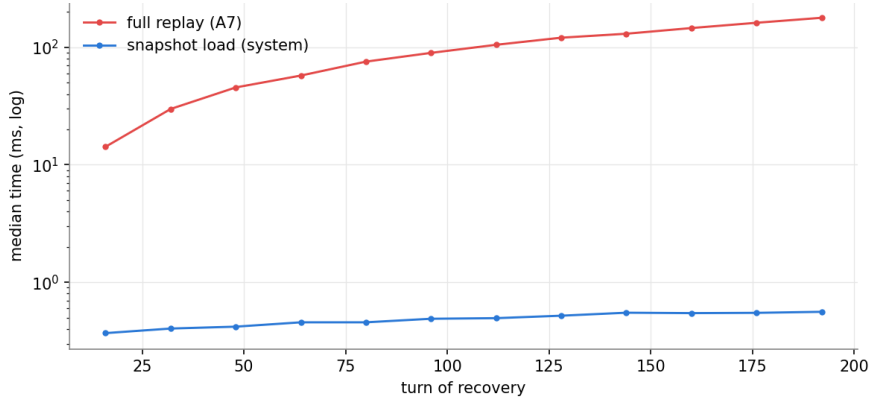


Figure 12: Median time to rebuild state at each snapshot point (log scale): full replay of the event log (A7) against snapshot loading (the system), over 42 sessions of 200 turns. The plot’s title strip, which lists the consistency rates of Table 14, is cropped.

7.7 Latency and token cost

Local work was negligible against generation (Table 15 and Figure 13). Validation took 0.039 ms at P95, well inside the 5 ms bound of H4, and the commit node (SQLite append, reduction, indexing) dominated local time at 18.4 ms. Single-stream LLM time, from the recall-prompt benchmark, was 6,906 ms at P50 and 9,409 ms at P95 for F4. Under the batched live stage (84 concurrent sessions) the median LLM time per F4 turn was about 89.6 s, a throughput setting rather than an interactive one. Per 100 turns F4 consumed 139,898 input and 15,394 output tokens, against 431,032 and 14,117 for full history (Table 7). Because the models were self-hosted, no monetary cost is reported.

Table 15: Per-stage latency (ms) of the live F4 graph, single stream (24 turns, two worlds). All 24 LLM responses in this stage came from the response cache, so the LLM row (0.09 ms at P50) is a cache lookup and is omitted.

	Input	Retrieval	Prompt	Parse	Validate	Commit	Output	Local total	End to end
P50	0.002	0.174	0.123	0.087	0.031	18.428	0.011	24.921	25.006
P95	0.003	0.310	0.158	0.118	0.039	21.049	0.013	27.526	27.609
P99	0.009	0.385	0.167	0.125	0.046	30.311	0.013	37.016	37.127

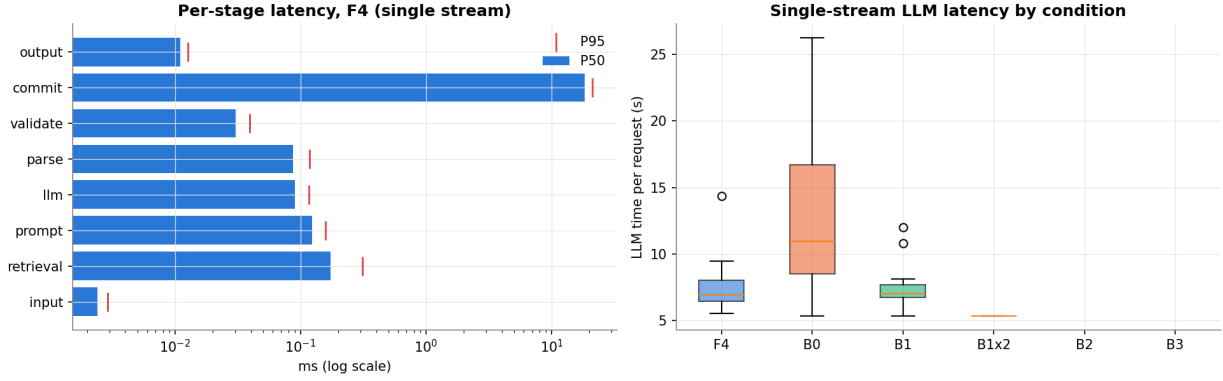


Figure 13: Left: P50 (bars) and P95 (ticks) latency of each F4 stage, single stream, log scale; the “llm” bar is a response-cache lookup (Table 15). Right: single-stream LLM time per request for each condition’s recall prompts; B2 and B3 were not reached before the stage deadline and B1×2 has one cached sample.

7.8 Generalisation across worlds and backbones

Both main effects held on every held-out world (Table 16 and Figure 14). By family, F4 minus B1 recall at $a \geq 20$ was +0.889 (W2), +0.796 (W3) and +0.898 (W4), and direct-minus-barrier attack success was +0.114, +0.086 and +0.121. F4 did not beat the best baseline on every world: B2 was ahead on W2c and B0 on W4-32, both within overlapping intervals. On the subset where both backbones ran (W1, W2a, W3, W4-32 at horizon 100; 72 F4 probes at $a \geq 20$), the 3.7B model kept F4’s recall close to the 7B model’s (81.9% against 86.1%) while every baseline dropped more: B0 from 79.2% to 61.1%, B2 from 84.7% to 73.6% and B3 from 70.8% to 48.6% (Figure 8, right). With the 3.7B model the barrier blocked every attack (0 of 280, against 18.6% under direct writes of the same outputs) and rejected no lawful twin (0 of 42 lawful proposals).

Table 16: Per-world and per-backbone breakdown. Best baseline for recall: highest of B0 to B3 on that world; for attacks: the baseline context with the lowest direct-write success. The 3.7B model ran at horizon 100 on four worlds with probes under the F4 context only, so its attack comparison is barrier against direct writes of the same outputs.

World	Backbone	Recall $a \geq 20$: F4	Recall: best baseline	Attack: F4 barrier	Attack: best baseline (direct)
W1	7B	87.0 [75.0, 97.7]	B2: 87.0 [80.3, 94.2]	7.1 [1.4, 14.3]	B0: 18.6 [10.0, 28.6]
W2a	7B	90.7 [87.1, 95.7]	B2: 85.2 [80.8, 90.0]	2.9 [0.0, 7.1]	B0: 17.1 [8.6, 25.7]
W2b	7B	92.6 [86.5, 98.1]	B2: 87.0 [77.8, 94.6]	4.3 [0.0, 10.0]	B0: 8.6 [2.9, 15.7]
W2c	7B	83.3 [75.0, 91.7]	B2: 87.0 [78.3, 96.3]	2.9 [0.0, 7.1]	B2: 10.0 [4.3, 17.1]
W3	7B	79.6 [71.4, 88.6]	B2: 74.1 [63.8, 83.9]	1.4 [0.0, 4.3]	B0: 11.4 [4.3, 20.0]
W4-32	7B	87.0 [76.0, 96.4]	B0: 90.7 [82.6, 96.8]	4.3 [0.0, 10.0]	B0: 15.7 [7.1, 24.3]
W4-128	7B	92.6 [85.7, 100.0]	B2: 88.9 [81.8, 95.8]	2.9 [0.0, 7.1]	B0: 14.3 [7.1, 22.9]
W1	3.7B	100.0 [100.0, 100.0]	B2: 88.9 [66.7, 100.0]	0.0 [0.0, 0.0]	F4 outputs: 22.9 [12.9, 32.9]
W2a	3.7B	83.3 [66.7, 100.0]	B2: 72.2 [66.7, 83.3]	0.0 [0.0, 0.0]	F4 outputs: 14.3 [7.1, 22.9]
W3	3.7B	66.7 [66.7, 66.7]	B2: 66.7 [50.0, 83.3]	0.0 [0.0, 0.0]	F4 outputs: 17.1 [8.6, 25.7]
W4-32	3.7B	77.8 [50.0, 100.0]	B0: 66.7 [50.0, 83.3]	0.0 [0.0, 0.0]	F4 outputs: 20.0 [11.4, 30.0]

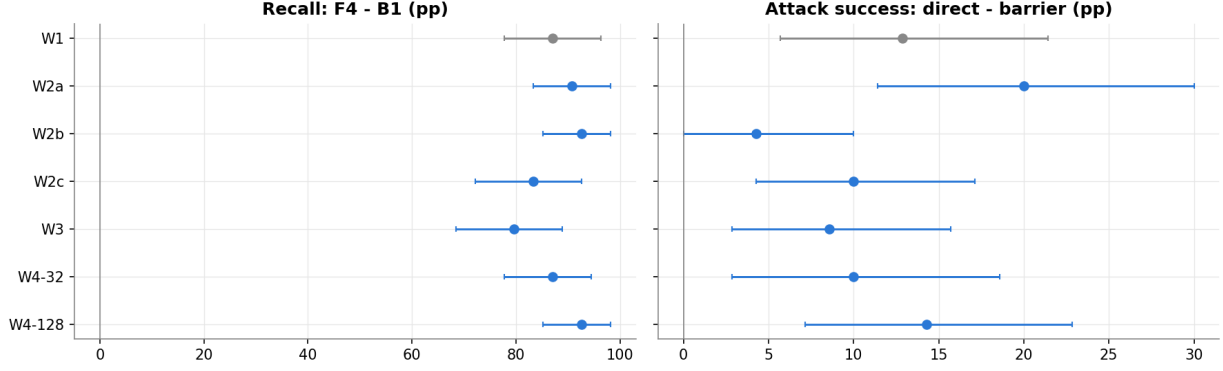


Figure 14: Per-world effects with 95% bootstrap intervals. Left: recall gain of F4 over B1 at fact age 20 or more. Right: reduction in attack success from direct writes to the barrier (F4 context). W1 (grey) is in-distribution; every held-out world favours the system on both effects.

7.9 Hypothesis tests

Table 17 summarises the verdicts. H1 fails on two counts: the gain over B2 is not significant, and F4 used more input tokens (1,399 per turn) than both B2 (1,102) and B3 (1,209), so “equal or lower token cost” is not met. H5 holds for the event-sourced design but not for the application’s load path. H6 fails because control affects recall and the two factors interact.

Table 17: Pre-registered hypotheses. Effects are differences in proportions (McNemar) or mean paired differences (Wilcoxon). p is Holm-corrected over the six confirmatory tests.

H	Test	n	Effect	p_{Holm}	Verdict
H1	recall $a \geq 20$, F4 vs B1 (McNemar, held-out)	324	+0.877	3.9×10^{-85}	not supported
	recall $a \geq 20$, F4 vs B2	324	+0.028	0.298	
	recall $a \geq 20$, F4 vs B3	324	+0.238	9.1×10^{-17}	
H2	MRR@6, F4 vs B2 (Wilcoxon, per probe, held-out)	540	+0.148	4.3×10^{-20}	supported
H3	attack success, direct vs barrier (McNemar, F4 context, all worlds)	490	+0.114	9.1×10^{-17}	supported
	live violations per transition, F3 vs F4 (Wilcoxon, per session)	14	+3.459	1.9×10^{-3}	
H4	false rejection $\leq 5\%$; validation P95 < 5 ms	—	0.0%; 0.039 ms	—	supported
H5	replay reproduces live state in 100% of event-sourced sessions	42 + live	100%	—	supported
H6	interaction small	—	recall $\beta_{MC} = 1.17$	—	not supported
H7	H1 and H3 directions on W2, W3, W4	—	all positive	—	supported

8 Error Analysis

Table 18 locates the failures. None of F4’s recall failures came from retrieval: in all 57 “hit but ignored” cases the gold memory was in the prompt and the model answered wrongly, and only 6 answers used the distractor’s value. Rolling contexts failed by forgetting (315 misses for B1), generic RAG by ignoring or confusing retrieved chunks (142 and 50), and the ablation without long-term memory by both (377 misses, 132 confusions). F4’s committed violations under the barrier (16) almost all coincide with validator gaps (17), which Table 10 places in the currency category. Desynchronisation counts were similar for F4, F2 and A6 (102, 101, 108), the three conditions that can reject proposals, which ties the effect to rejection without regeneration rather than to memory.

Table 18: Automatically coded failures (recall categories on all worlds; up to 50 examples per category and condition are exported to [annotation/error_samples.csv](#)).

Category	F4	B0	B1	B1×2	B2	B3	A1	A3	F2	A6
Retrieval miss (gold not in prompt)	0	0	315	202	5	11	377	6	–	–
Hit but ignored	57	106	27	47	70	142	11	74	–	–
Confused with distractor	6	26	89	78	1	50	132	1	–	–
Schema failure	6	7	4	2	19	9	3	6	–	–
Validator gap (probes)	17	13	13	–	13	15	–	–	–	–
Oracle violation under barrier (probes)	16	13	13	–	13	14	–	–	–	–
Desynchronisation (live, proxy)	102	–	–	–	–	–	–	–	101	108

9 Discussion

What is genuinely new. The components are standard; the evidence is not. Three findings are, to our knowledge, not available from prior work. (i) *Paired attribution of protection*: committing identical outputs three ways shows that, of the 13.3-point reduction in attack success from direct writes to the barrier, 9.1 points come from the reducer’s own semantics and 4.2 from the validator; validator and reducer must be evaluated as a pair. (ii) *Oracle-revealed blind spots*: an oracle written from the specification exposed a currency rule that the validator never implemented, which no validator-scored metric could have revealed. (iii) *Measured costs of control*: zero false rejections, but a five- to seven-fold rise in desynchronisation (from about 3% to about 19% of live turns) and a load path that silently loses up to 15 turns of logged events (8 on average in our crash simulation).

Memory. Scoped long-term memory is what makes long-range recall possible: without it recall beyond the 8-turn buffer is zero, and a rolling context of equal budget forgets every fact older than 10 turns. Against a strong flat vector store the advantage narrows to a non-significant 2.8 points at a 27% higher token cost. Scoping’s measurable benefits were ranking (MRR), distractor exclusion and stability across seeds and backbones. The benchmark always probes a fact with the NPC it was told to, at that NPC’s location, which favours scoping and gives the fallback little to do; the value of the buffer and fallback in free play, where facts are recalled elsewhere, is not measured here. Proposition 2 identifies a further risk at scale: once a store holds many similar out-of-scope entries, post-filtering the global top-30 can drop in-scope memories, which a filtered index avoids [26].

State control. The barrier works where it has a rule and is only as good as its rule set. Bounding currency gains, validating per turn rather than per proposal, and checking NPC relocation would close the gaps observed. Blocking without regenerating trades structural integrity for narrative consistency; regenerating or annotating the reply on rejection is the obvious fix and carries a latency cost.

Implications for practitioners. Treat LLM output as untrusted input to a state machine; keep the reducer defensive even behind a validator; audit with a checker that does not share the validator’s code; and recover by snapshot plus suffix, never by snapshot alone.

10 Limitations and Threats to Validity

1. **Sample size.** Level 2 of the ladder (63 recall sessions, 490 attacks, 14 live sessions per cell) is well below the level-7 targets; live intervals are wide.
2. **Scoring.** Recall, leaks and desynchronisation are scored automatically (key words, secret key-words, refusal and arrival cues). Blinded sheets for two raters (1,250 recall, 150 persona and

lore, 240 desynchronisation items) are exported, but no human labels, Cohen’s κ or LLM-judge agreement exist yet.

3. **Worlds.** W1 is the development world, W4 is templated, and W3 was authored by the notebook author rather than an independent contributor.
4. **Scripted players.** Fixed scripts remove adaptation confounds but do not react to NPCs; player experience needs a user study.
5. **Backbones.** Two self-hosted models of one family with capped thinking; the game’s default hosted backbone was not evaluated, and its latency is not reproduced.
6. **Baselines.** B4 (scripted FSM) and B5 (MemGPT-style memory) were not run, and the baselines may be weaker than tuned production systems; their code and prompts are in the notebook.
7. **Secrets.** The model never receives NPC secrets, so leak rates do not measure secret keeping.
8. **Violation metric.** Equation (6) counts flagged turns over transitions; persistent corruption inflates direct-write rates, so live violation rates compare conditions but are not counts of distinct violations.
9. **Memory scale.** Exact search with pruning at 1,000 entries; campaigns of thousands of turns need summarisation or hierarchical memory.

Threats to validity. *Internal:* inputs, prompts, schemas and backbones are identical across conditions, request seeds depend only on the probe or turn, and probe outputs are committed in all three modes, so control comparisons are exactly paired. *Construct:* violations are measured by an independent oracle, but its O8 bound (+50) is a design choice the system never encoded; recall is key-word matched on invented values. *External:* claims are restricted to the tested worlds and backbones. *Conclusion:* hypotheses were stated in advance, the confirmatory family is Holm-corrected, and both unsupported hypotheses are reported. The factorial p -values are approximate posterior summaries from a variational fit.

11 Conclusion

LLM NPCs need to remember what happened, respect the rules of the world, and do both at interactive cost. We tested an engine that pairs scoped dual-tier memory with a validated, event-sourced state against independent baselines, in a factorial design with ablations, on seven worlds, with adversarial probes scored by an oracle that shares no code with the validator. Three claims are supported. Long-term retrieval scoped to the current place and person keeps recall high and flat over 160 turns where a rolling context of equal size forgets everything after ten, and it ranks the right memory higher than flat vector memory. A pre-commit barrier over an event-sourced state blocked every movement, fabrication, theft and injection attack the model attempted without rejecting lawful requests, and the event log reproduced the live state exactly. Both effects held on every held-out world family and on a smaller backbone. Two claims are not supported: the memory design is not clearly better than flat vector memory at its token cost, and memory and control are not independent. The paired, oracle-audited protocol also exposed concrete defects (unbounded currency gains, a lossy load path, persistent corruption without the reducer, and dialogue that contradicts rejected changes) that a validator-scored evaluation would have missed.

Future work. Fix the defects found (bound currency gains and validate per turn; fold the logged suffix on load; event-source the active NPC; align the prompt’s trust range; pass secrets above the trust threshold); regenerate dialogue on rejection; rerun at the full sample size with an independently authored W3, human annotation, the hosted backbone and other model families, adding B4 and B5; probe recall away from the place a fact was told; validate narrative plausibility, not only structural legality; and evaluate filtered or hierarchical indices for long campaigns.

Data and licences. W2 content derives from LIGHT under CC BY-NC 4.0, so the released W2 worlds are for non-commercial use. Both evaluated backbones are Apache 2.0.

References

- [1] Roberto Gallotta, Graham Todd, Marvin Zammit, Sam Earle, Antonios Liapis, Julian Togelius, and Georgios N. Yannakakis. Large language models and games: A survey and roadmap. *IEEE Transactions on Games*, 2024. doi:[10.1109/TG.2024.3461510](https://doi.org/10.1109/TG.2024.3461510).
- [2] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi:[10.1162/tacl_a_00638](https://doi.org/10.1162/tacl_a_00638).
- [3] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024. doi:[10.18653/v1/2024.acl-long.747](https://doi.org/10.18653/v1/2024.acl-long.747).
- [4] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2410.10813>.
- [5] Ruoyao Wang, Graham Todd, Ziang Xiao, Xingdi Yuan, Marc-Alexandre Côté, Peter Clark, and Peter Jansen. Can language models serve as text-based world simulators? In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 1–17, 2024. doi:[10.18653/v1/2024.acl-short.1](https://doi.org/10.18653/v1/2024.acl-short.1).
- [6] Najoung Kim and Sebastian Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, 2023. doi:[10.18653/v1/2023.acl-long.213](https://doi.org/10.18653/v1/2023.acl-long.213).
- [7] Yingpeng Ma, Jianhao Yan, Bei Shi, Ka Hou Kam, Runnan Wang, Xuebo Liu, Yulong Chen, Yue Zhang, and Derek F. Wong. Can LLM agents stick to the script? A benchmark for long-horizon consistency in interactive narratives. [arXiv:2608.08160](https://arxiv.org/abs/2608.08160); accepted at ICML 2026, 2026.
- [8] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, pages 1–22, 2023. doi:[10.1145/3586183.3606763](https://doi.org/10.1145/3586183.3606763).
- [9] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2303.11366>.
- [10] Jack Urbanek, Angela Fan, Siddharth Karamcheti, Saachi Jain, Samuel Humeau, Emily Dinan, Tim Rocktäschel, Douwe Kiela, Arthur Szlam, and Jason Weston. Learning to speak and act in a fantasy text adventure game. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 673–683, 2019. doi:[10.18653/v1/D19-1062](https://doi.org/10.18653/v1/D19-1062).
- [11] Prithviraj Ammanabrolu, Jack Urbanek, Margaret Li, Arthur Szlam, Tim Rocktäschel, and Jason Weston. How to motivate your dragon: Teaching goal-driven agents to speak and act in fantasy worlds. In *Proceedings of the 2021 Conference of the North American Chapter of the*

Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), pages 807–833, 2021. doi:[10.18653/v1/2021.naacl-main.64](https://doi.org/10.18653/v1/2021.naacl-main.64).

- [12] Marc-Alexandre Côté, Ákos Kádár, Xingdi Yuan, Ben Kybartas, Tavian Barnes, Emery Fine, James Moore, Matthew Hausknecht, Layla El Asri, Mahmoud Adada, Wendy Tay, and Adam Trischler. TextWorld: A learning environment for text-based games. In *Computer Games (CGW 2018)*, volume 1017 of *Communications in Computer and Information Science*, pages 41–75. Springer, 2019. doi:[10.1007/978-3-030-24337-1_3](https://doi.org/10.1007/978-3-030-24337-1_3).
- [13] Matthew Hausknecht, Prithviraj Ammanabrolu, Marc-Alexandre Côté, and Xingdi Yuan. Interactive fiction games: A colossal adventure. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pages 7903–7910, 2020. doi:[10.1609/aaai.v34i05.6297](https://doi.org/10.1609/aaai.v34i05.6297).
- [14] Christopher Zhang Cui, Xingdi Yuan, Ziang Xiao, Prithviraj Ammanabrolu, and Marc-Alexandre Côté. TALES: Text adventure learning environment suite. [arXiv:2504.14128](https://arxiv.org/abs/2504.14128), 2025.
- [15] Chris Callison-Burch, Gaurav Singh Tomar, Lara J. Martin, Daphne Ippolito, Suma Bailis, and David Reitter. Dungeons and dragons as a dialog challenge for artificial intelligence. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 9379–9393, 2022. doi:[10.18653/v1/2022.emnlp-main.637](https://doi.org/10.18653/v1/2022.emnlp-main.637).
- [16] Nader Akoury, Shufan Wang, Josh Whiting, Stephen Hood, Nanyun Peng, and Mohit Iyyer. STORIUM: A dataset and evaluation platform for machine-in-the-loop story generation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6470–6484, 2020. doi:[10.18653/v1/2020.emnlp-main.525](https://doi.org/10.18653/v1/2020.emnlp-main.525).
- [17] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. [arXiv:2310.08560](https://arxiv.org/abs/2310.08560), 2023.
- [18] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024. doi:[10.1609/aaai.v38i17.29946](https://doi.org/10.1609/aaai.v38i17.29946).
- [19] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi:[10.52202/079017-1902](https://doi.org/10.52202/079017-1902).
- [20] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-Mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. doi:[10.52202/085713-0593](https://doi.org/10.52202/085713-0593).
- [21] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. [arXiv:2504.19413](https://arxiv.org/abs/2504.19413), 2025.
- [22] Theodore R. Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas L. Griffiths. Cognitive architectures for language agents. *Transactions on Machine Learning Research*, 2024. URL <https://arxiv.org/abs/2309.02427>.
- [23] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025. doi:[10.1145/3748302](https://doi.org/10.1145/3748302).
- [24] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. URL <https://arxiv.org/abs/2005.11401>.

- [25] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [26] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023 (WWW)*, pages 3406–3416, 2023. doi:[10.1145/3543507.3583552](https://doi.org/10.1145/3543507.3583552).
- [27] Subbarao Kambhampati, Karthik Valmeekam, Lin Guan, Mudit Verma, Kaya Stechly, Siddhant Bhambri, Lucas Paul Saldyt, and Anil B. Murthy. Position: LLMs can’t plan, but can help planning in LLM-modulo frameworks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 22895–22907, 2024. URL <https://proceedings.mlr.press/v235/kambhampati24a.html>.
- [28] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 10932–10952, 2023. doi:[10.18653/v1/2023.emnlp-main.674](https://doi.org/10.18653/v1/2023.emnlp-main.674).
- [29] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you’ve signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023. doi:[10.1145/3605764.3623985](https://doi.org/10.1145/3605764.3623985).
- [30] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [31] Steph Buongiorno, Lawrence Klinkert, Zixin Zhuang, Tanishq Chawla, and Corey Clark. PANGeA: Procedural artificial narrative using generative AI for turn-based, role-playing video games. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, volume 20, pages 156–166, 2024. doi:[10.1609/aiide.v20i1.31876](https://doi.org/10.1609/aiide.v20i1.31876).
- [32] Minkyung Kim, Junsik Kim, Woongcheol Yang, Sangdon Park, and Sohee Bae. Aligning large language models with procedural rules: An autoregressive state-tracking prompting for in-game trading. [arXiv:2510.25014](https://arxiv.org/abs/2510.25014), 2025.
- [33] Martin Braas and Lukas Esterle. Fixed-persona SLMs with modular memory: Scalable NPC dialogue on consumer hardware. [arXiv:2511.10277](https://arxiv.org/abs/2511.10277), 2025.
- [34] Vanessa Figueiredo and David Elumeze. Symbolically scaffolded play: Designing role-sensitive prompts for generative NPC dialogue. [arXiv:2510.25820](https://arxiv.org/abs/2510.25820), 2025.
- [35] Zimu Xu. Long-lived characters, local inference: Incremental memory maintenance for game NPCs. [arXiv:2609.18935](https://arxiv.org/abs/2609.18935), 2026.
- [36] Yi-Chun Chen. Reconstructing persistent worlds from narratives for narrative-grounded interactive experiences. [arXiv:2608.04037](https://arxiv.org/abs/2608.04037), 2026.
- [37] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3980–3990, 2019. doi:[10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410).

- [38] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, pages 5776–5788, 2020. URL <https://arxiv.org/abs/2002.10957>.
- [39] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi:[10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).
- [40] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi:[10.1214/aos/1176344552](https://doi.org/10.1214/aos/1176344552).
- [41] Christopher A. Field and Alan H. Welsh. Bootstrapping clustered data. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 69(3):369–390, 2007. doi:[10.1111/j.1467-9868.2007.00593.x](https://doi.org/10.1111/j.1467-9868.2007.00593.x).
- [42] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi:[10.1007/BF02295996](https://doi.org/10.1007/BF02295996).
- [43] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi:[10.2307/3001968](https://doi.org/10.2307/3001968).
- [44] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [45] Skipper Seabold and Josef Perktold. Statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010. doi:[10.25080/Majora-92bf1922-011](https://doi.org/10.25080/Majora-92bf1922-011).
- [46] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. doi:[10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).

A Sample-Size Ladder

Table 19: Pre-declared sample-size levels and the planner’s estimated hours ([metrics/budget_plan.csv](#)). Level 2 was run.

Level	Sessions/(world, H)	Probes/(world, cat.)	Live/world	Recall (h)	Probes (h)	Live (h)	Seed var. (h)	Total (h)
0	1	4	1	0.84	0.70	0.99	0.66	3.66
1	2	6	1	1.73	1.08	0.99	0.66	5.13
2	3	10	2	2.63	1.84	1.98	0.66	8.18
3	4	14	2	3.53	2.60	1.98	0.66	10.09
4	6	20	3	5.33	3.75	1.98	0.66	13.48
5	10	30	4	8.92	5.65	2.97	0.66	20.94
6	20	45	6	17.89	8.51	3.97	0.66	35.68
7	40	60	10	35.84	11.37	6.93	0.66	63.03

B Audit Notes

Auditing the code and raw outputs against the earlier project article ([docs/research_article.md](#)) found the following. All are corrected in this paper, and the article and the repository README have since been updated to match.

- The chosen ladder level, index 2 of levels 0 to 7, is the *third* of eight levels, not the second.
- Live probes have recorded fact ages of 30 to 34 turns (`age` column of `predictions/live_turns.csv`), not about 20 to 32; the configured `LIVE_FACT_AGES` are not used by the script generator.
- Theft has lawful twins for 50 of 70 attacks per context (71%).
- Live violation rates above 100 per 100 transitions arise because the oracle re-flags persistent corruption on later turns (Equation (6)), not because one transition breaks several invariants: the numerator counts flagged turns.
- The factorial model is a variational-Bayes mixed GLM; its “SE” and “*p*” are posterior summaries (Section 5.8).

All other values in the earlier article match the run outputs.